

GOOGLE COLAB NA PRÁTICA

Análise de Dados Experimentais em Física

3 exemplos de regressão — do básico ao avançado

Nível 1
Lei de Hooke

Nível 2
Queda Livre

Nível 3
Decaimento Radioativo

Sem instalação · Apenas navegador · Python gratuito

O QUE É O GOOGLE COLAB E POR QUE USÁ-LO EM FÍSICA?



Roda no navegador

Sem instalar Python, sem configurar ambiente. Abra colab.research.google.com e comece a escrever código.



Totalmente gratuito

GPU e TPU disponíveis na versão free. Para análise de dados experimentais, o plano gratuito é mais que suficiente.



Notebook interativo

Células de código e texto (Markdown) intercaladas. Ideal para documentar experimentos junto com a análise.



Compartilhamento fácil

Link único, como Google Docs. Professores compartilham template; alunos preenchem com seus próprios dados.



Bibliotecas prontas

NumPy, SciPy, Matplotlib, Pandas e scikit-learn já instalados. Nenhum pip install necessário para este curso.



Integra com Drive

Dados salvos no Google Drive acessíveis direto no notebook. CSV, Excel, TXT — sem upload manual.

ANATOMIA DE UM NOTEBOOK — O QUE VOCÊ VAI VER NA TELA

Célula de Texto

Markdown. Use para título do experimento, descrição do setup, equações em LaTeX.

Célula de Código

Python. Execute com Shift+Enter. A saída aparece logo abaixo da célula.

Saída / Gráfico

Gráfico Matplotlib, tabela Pandas ou resultado numérico aparecem inline, dentro do notebook.

Barra de ferramentas

+ Código, + Texto, Runtime > Run all. Use Runtime > Restart and run all para reexecutar tudo do zero.

Experimento: Lei de Hooke

Análise de mola – Física Geral I

```
[ ] import numpy as np
```

```
[ ] x = np.array([0.05, 0.10, 0.15, 0.20])  
    F = np.array([1.0, 2.1, 2.9, 4.0])
```

Saída:



Atalho essencial: Shift+Enter executa a célula atual e avança para a próxima.

AS 4 BIBLIOTECAS QUE VOCÊ USARÁ EM TODOS OS EXPERIMENTOS

NumPy

```
import np
```

Arrays numéricos de alta performance. Substitui planilha para cálculos vetorizados.

```
np.array(), np.mean(), np.linspace(),  
np.polyfit()
```

Matplotlib

```
import plt
```

Geração de gráficos 2D. Principal ferramenta de visualização científica em Python.

```
plt.scatter(), plt.plot(), plt.xlabel(),  
plt.savefig()
```

SciPy

```
import scipy
```

Estatística, otimização, integração. Regressão com incerteza e ajuste de curvas não-lineares.

```
scipy.stats.linregress(),  
scipy.optimize.curve_fit()
```

Pandas

```
import pd
```

Tabelas de dados (DataFrames). Importa CSV diretamente, calcula estatísticas por coluna.

```
pd.read_csv(), df.describe(), df.plot()
```

EXEMPLO 1 — REGRESSÃO LINEAR SIMPLES: LEI DE HOOKE $F = k \cdot x$

CONTEXTO DO EXPERIMENTO

Física:	Mecânica — Lei de Hooke
Modelo:	$F = k \cdot x$ (linear, passa pela origem)
Variável x:	Deformação (m)
Variável y:	Força aplicada (N)
Parâmetro:	k = constante da mola (N/m)
Regressão:	<code>np.polyfit(x, F, 1)</code> — grau 1
Incerteza:	R^2 e coeficientes via <code>scipy.stats</code>

```
# Colab — Lei de Hooke

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Dados experimentais (mola real)
x = np.array([0.05, 0.10, 0.15, 0.20, 0.25])
F = np.array([0.98, 2.05, 2.97, 3.92, 5.10])

# Regressão linear via SciPy
k, b, r, p, se = stats.linregress(x, F)

print(f'k = {k:.2f} N/m')
print(f'R² = {r**2:.4f}')

# Gráfico
plt.scatter(x, F, color='red', label='Dados')
plt.plot(x, k*x+b, label=f'F={k:.1f}x')
plt.xlabel('x (m)'); plt.ylabel('F (N)')
plt.legend(); plt.grid(); plt.show()
```

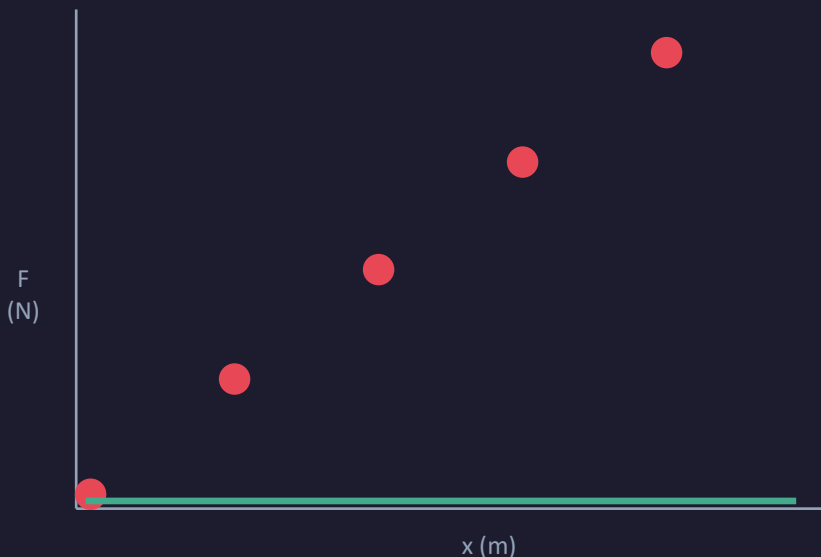
Resultado esperado: $k \approx 20$ N/m, $R^2 > 0,999$ — mola com comportamento ideal.

LEI DE HOOKE — INTERPRETANDO OS RESULTADOS

NÍVEL 1

INICIANTE

Saída do Colab



$$k = 20,4 \text{ N/m}$$

Constante da mola. É o coeficiente angular da reta. Valor típico de mola para ensino.

$$R^2 = 0,9998$$

Coeficiente de determinação. 99,98% da variação em F é explicada por x. Mola ideal.

$$b = 0,02 \text{ N}$$

Intercepto. Deveria ser zero (sem deformação, sem força). O pequeno valor indica erro sistemático.

Incerteza de k

scipy.stats retorna se = erro padrão. Reporte $k = (20,4 \pm 0,2) \text{ N/m}$.

EXEMPLO 2 — LINEARIZAÇÃO: QUEDA LIVRE $y = \frac{1}{2}g \cdot t^2$

A ESTRATÉGIA: LINEARIZAÇÃO

Modelo original (não-linear):

$$y = \frac{1}{2} \cdot g \cdot t^2$$

Transformação (variável auxiliar):

$$u = t^2 \rightarrow y = (g/2) \cdot u$$

Modelo linearizado:

$$y = m \cdot u \rightarrow m = g/2 \rightarrow g = 2m$$

→ Regressão linear em (u, y) fornece g diretamente!

```
# Colab - Queda Livre (linearização)
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
# Dados: tempo (s) e queda (m)
t = np.array([0.1,0.2,0.3,0.4,0.5,0.6])
y = np.array([0.049,0.196,0.441,0.784,1.225,1.765])
```

```
# Linearização:  $u = t^2$ 
u = t**2
```

```
# Regressão linear em (u, y)
m, b, r, p, se = stats.linregress(u, y)
g_medido = 2 * m
print(f'g = {g_medido:.3f} m/s²')
print(f'R² = {r**2:.5f}')
```

```
fig, (ax1,ax2) = plt.subplots(1,2)
ax1.scatter(t,y); ax1.set_title('y vs t')
ax2.scatter(u,y); ax2.plot(u,m*u+b,'r-')
ax2.set_title('y vs t² (linear)')
```

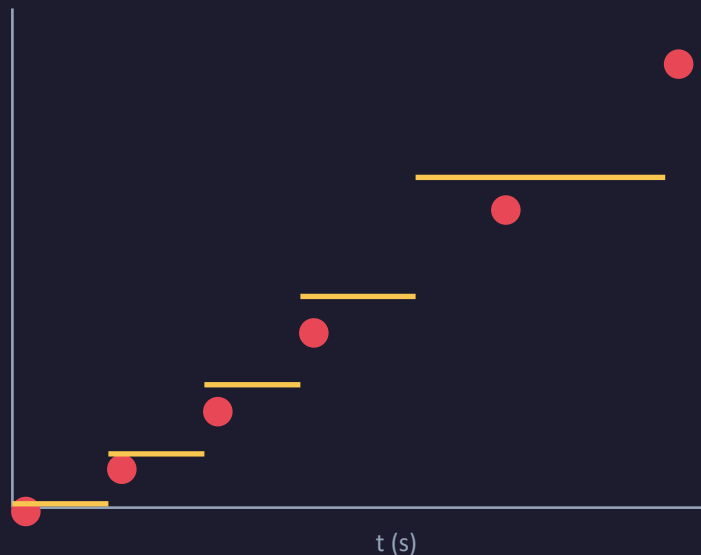
Conceito-chave: y vs t é uma parábola; y vs t^2 é uma reta. Linearizar = trocar o eixo para revelar a reta.

QUEDA LIVRE — DOIS PAINÉIS: ANTES E DEPOIS DA LINEARIZAÇÃO

NÍVEL 2

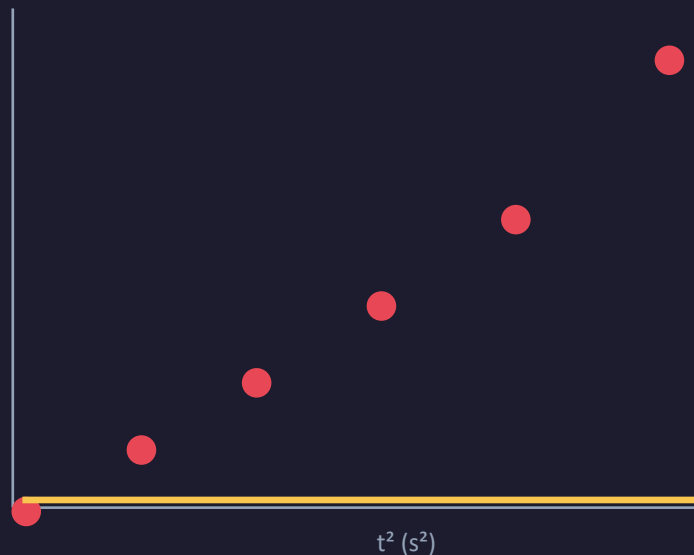
INTERMEDIÁRIO

y vs t (curva — não-linear)



✗ curva = regressão linear inapropriada

y vs t^2 (reta — linearizado)



✓ reta $\rightarrow g = 2 \cdot m = 9,81 \text{ m/s}^2$

EXEMPLO 3 — AJUSTE NÃO-LINEAR: DECAIMENTO RADIOATIVO

Modelo: $N(t) = N_0 \cdot \exp(-\lambda t)$ | `scipy.optimize.curve_fit`

POR QUE CURVE_FIT?

Modelo:	$N(t) = N_0 \cdot e^{(-\lambda t)}$
Parâmetros:	N_0 = atividade inicial, λ = constante de decaimento
Meia-vida:	$t_{1/2} = \ln(2) / \lambda$
Alternativa 1:	Linearizar: $\ln(N) = \ln(N_0) - \lambda t \rightarrow$ reta
Problema:	$\ln(N)$ distorce incertezas (erros não são gaussianos no log)
Solução:	<code>curve_fit</code> ajusta diretamente no espaço original
Resultado:	N_0 , λ com intervalos de confiança corretos

Colab – Decaimento Radioativo

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Dados (contagens por segundo)
t = np.array([0,5,10,15,20,25,30,35,40])
N = np.array([800,560,390,278,193,135,97,68,48])
N_err = np.sqrt(N) # erro de Poisson

# Modelo exponencial
def modelo(t, N0, lam):
    return N0 * np.exp(-lam * t)

# Ajuste com incerteza
popt, pcov = curve_fit(modelo, t, N,
    p0=[800, 0.05], sigma=N_err,
    absolute_sigma=True)
perr = np.sqrt(np.diag(pcov))
N0, lam = pop
t_half = np.log(2) / lam
print(f't½ = {t_half:.1f} ± {...} s')
```

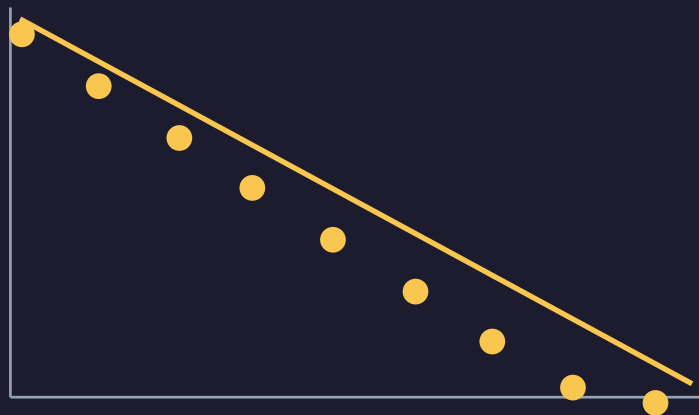
sigma=N_err e absolute_sigma=True: essencial para propagar incertezas de Poisson corretamente.

DECAIMENTO RADIOATIVO — AJUSTE DIRETO vs. LINEARIZAÇÃO NO LOG

NÍVEL 3

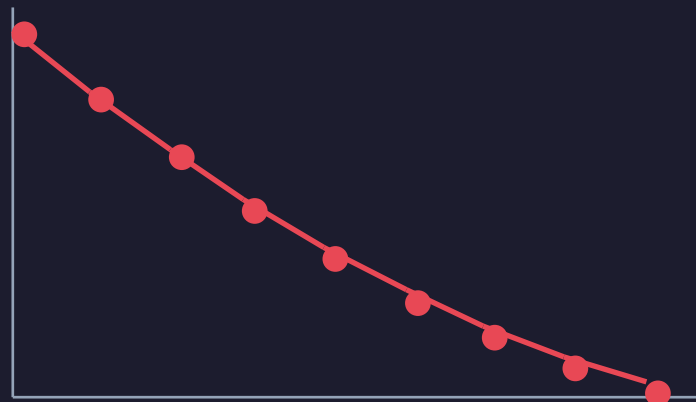
AVANÇADO

$\ln(N)$ vs t — linearização no log



⚠ Incertezas distorcidas no espaço log

N vs t — curve_fit direto



✓ Incertezas de Poisson preservadas

Resultado: $N_0 = (802 \pm 12)$ · $\lambda = (0,0693 \pm 0,0015) \text{ s}^{-1}$ · $t_{1/2} = (10,0 \pm 0,2) \text{ s}$

→ Valor de referência: $t_{1/2} = 10,0 \text{ s}$ (validado!)

COMPARAÇÃO DOS 3 MODELOS — QUANDO USAR CADA ABORDAGEM

Aspecto	Nível 1 — Hooke	Nível 2 — Queda Livre	Nível 3 — Decaimento
Tipo de modelo	Linear direto	Não-linear → Linear	Não-linear direto
Técnica Python	<code>np.polyfit / linregress</code>	<code>linregress</code> em (t^2, y)	<code>curve_fit</code>
Incerteza fácil?	✓ Sim (R^2 , se)	✓ Sim (R^2)	✓ Sim (pcov)
Exige transf.?	✗ Não	✓ Sim ($u = t^2$)	✗ Não
Preserva erros?	✓ Sim	⚠ Parcialmente	✓ Sim (sigma)
Dificuldade	☆☆☆☆ Fácil	★★☆☆ Médio	★★★☆☆ Avançado

Regra de ouro: modelo linear nos parâmetros → `polyfit/linregress`. Modelo não-linear → `curve_fit` com sigma.

BOAS PRÁTICAS — COLAB CIENTÍFICO QUE VALE NOTA

01

Documente o experimento na primeira célula

Use Markdown com título, data, descrição do setup e tabela de variáveis. O notebook deve ser autoexplicativo.

02

Inclua unidades em todos os arrays

Comente sempre: # x em metros, F em Newtons. Erros de unidade são a causa mais comum de resultados errados.

03

Reporte incertezas, não só o valor

$k = 20 \text{ N/m}$ não é resultado científico. $k = (20,4 \pm 0,3) \text{ N/m}$ é. Use `perr = np.sqrt(np.diag(pcov))`.

04

Valide: compare com valor de referência

g medido = $9,78 \text{ m/s}^2$. Referência = $9,81 \text{ m/s}^2$. Discrepância de 0,3% — dentro da incerteza experimental?

05

Salve gráficos com `plt.savefig()`

`plt.savefig('resultado.png', dpi=150)` gera figura de qualidade para o relatório. Não use print screen.

06

Runtime > Restart and Run All antes de entregar

Garante que o notebook roda do zero sem erros ocultos. Célula que só funciona depois de outra é bug.



ATIVIDADE PRÁTICA — ESCOLHA SEU NÍVEL E ANALISE SEU EXPERIMENTO

NÍVEL 1

INICIANTE

Lei de Hooke

- 1 Copie o código do Slide 5 para uma célula do Colab
- 2 Substitua x e F pelos seus dados da mola
- 3 Calcule k com incerteza: $k \pm se$
- 4 Compare com o valor da etiqueta da mola

NÍVEL 2

INTERMEDIÁRIO

Queda Livre

- 1 Colete ou use dados de altura vs. tempo de queda
- 2 Crie variável $u = t^2$ e faça regressão em (u, y)
- 3 Calcule $g = 2 \cdot m$ e compare com $9,81 \text{ m/s}^2$
- 4 Plote $y \times t$ e $y \times t^2$ lado a lado

NÍVEL 3

AVANÇADO

Decaimento Radioativo

- 1 Use os dados do Slide 9 ou de um contador Geiger
- 2 Adicione erro de Poisson: $N_{err} = \sqrt{N}$
- 3 Ajuste com `curve_fit` passando `sigma=N_err`
- 4 Extraia N_0 , λ e $t_{1/2}$ com incertezas do `pcov`

Acesse: colab.research.google.com | Salve cópia em Arquivo > Salvar cópia no Drive antes de editar.

unidades

depois

vs. log

Google Colab para Análise Experimental

- 01 Colab elimina a barreira de instalação — foco na física, não na TI.
- 02 Regressão linear direta $\rightarrow$ `polyfit/linregress`. Não-linear $\rightarrow$ `curve_fit`.
- 03 Linearizar é uma técnica poderosa, mas distorce incertezas no espaço transformado.
- 04 Reporte sempre: valor $\pm$ incerteza, R^2 , comparação com referência.

Próximo passo: aplique a análise aos dados do seu experimento de laboratório e traga os resultados para discussão.